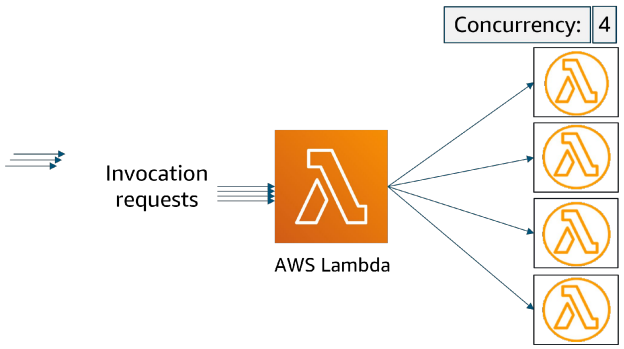
Lambda functions run in on-demand ephemeral environments

Concurrency: The number of function invocations that are running at one time



aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Each invocation of a Lambda function runs in an ephemeral environment—one that is initialized on demand, lasts a brief time, and then is removed. Lambda manages creating and tearing down these environments for your functions. You don't have direct control over these environments.

Concurrency is the number of Lambda function invocations that are running at one time. Lambda provides the environments on demand as it receives requests, which is how Lambda quickly scales horizontally.

Factors that influence concurrency include the following:

- **Reserved concurrency** on a function – This optional value is set per function and reserves a subset of the Regional quota for the function. It also establishes the maximum concurrent instances that are permitted for the function. For more information about managing reserved concurrency, see "Configuring reserved concurrency" at https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html.
- **Regional quota** – This quota is the total number of invocations that can run concurrently across all Lambda functions within an account by Region. AWS sets this number as a soft quota on the account. For more information about quotas, see "Lambda Quotas" at https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html.
- **Burst quota** – Each Region has a burst limit, which prevents concurrency from increasing too quickly if a large spike of requests occurs in a short time period. You cannot modify this limit. For more information about bursts, see "Lambda function scaling" at https://docs.aws.amazon.com/lambda/latest/dg/invocation-scaling.html.
- **Request rate and function duration** – This value is the rate at which new invocation requests for the function arrive, in conjunction with how long Lambda takes to run the function.
- **The event source** – Lambda manages concurrency of requests in different ways based on the type of event that invokes it.

To see how these concepts interact, consider how concurrency works under the hood:

Concurrency = Request Rate (requests/sec) * Average Execution Duration (seconds)

The Scenario & Base Parameters

Suppose your AWS account is operating in an AWS Region with the following settings:

- **Regional Quota:** 1,000 concurrent executions across your entire account in that region.
- **Burst Quota:** Initial burst limit (typically between 500 and 3,000 depending on the region, e.g., 1,000), followed by a scaling rate of 1,000 additional instances every 10 seconds per function.
- **Function A (Weather API):** Configured with a **Reserved Concurrency** of 200.
- **Other Functions (B, C, D):** No reserved concurrency set; they draw from the shared unreserved pool.

Step-by-Step Breakdown by Example

1. Request Rate and Function Duration

- **Condition:** Function A receives **400 requests per second (RPS)**, and each request takes **0.5 seconds** (500 ms) to execute.
- **Calculated Concurrency Demand:**
  400 * 0.5 = 200 concurrent instances
- **Behavior:** Function A needs 200 execution environments running at the exact same millisecond.

2. Reserved Concurrency on a Function

- **What it means:** A dedicated ceiling and floor for a specific function carved out of the Regional pool.
- **Behavior:**
  - Function A is guaranteed up to **200 concurrent executions**. No other functions in your account can touch these 200 slots, even if Function A is completely idle.
  - It also acts as a **hard ceiling**. If Function A's traffic suddenly doubles to 800 RPS 800 * 0.5 = 400 concurrency demand, Lambda will serve 200 instances and throttle the remaining 200 with ***429 Too Many Requests***. This protects your

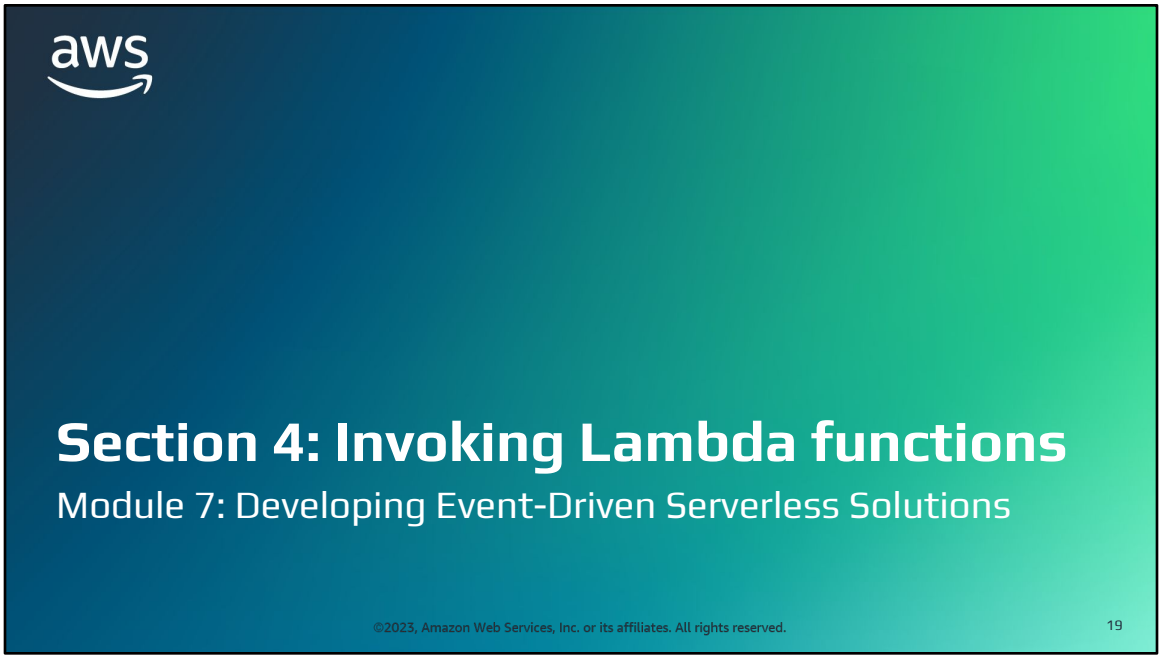
- downstream database from being overwhelmed.

3. Regional Quota

- **What it means:** The overall ceiling for the sum of all concurrent Lambda invocations across all functions in that Region.
- **Behavior:**
  - Total Account Quota: 1,000.
  - Function A reserves: 200.
  - Remaining unreserved pool: 1,000 - 200 = 800 available for Functions B, C, and D.
  - If Function B experiences a traffic spike needing 900 concurrent executions, it can only scale up to 800. The remaining 100 requests will throttle because the regional unreserved pool is exhausted. However, **Function A is completely unaffected** and continues to serve its 200 requests because its concurrency was isolated via reserved concurrency.

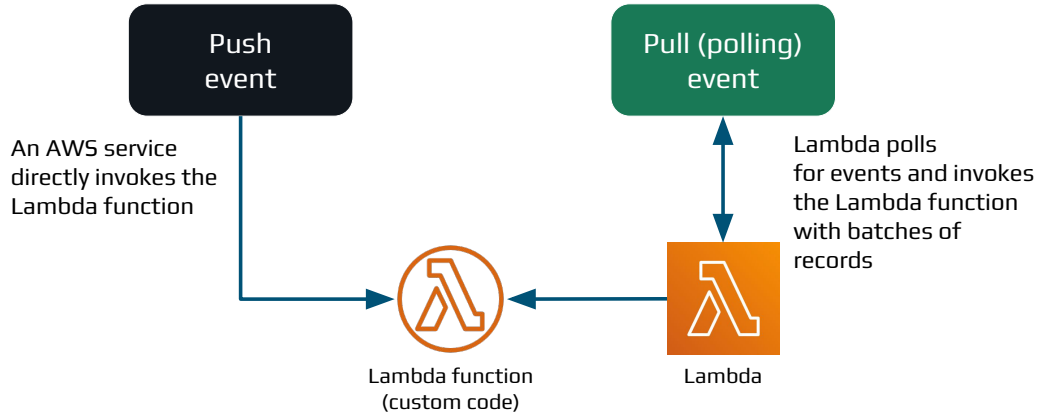
4. Burst Quota

- **What it means:** The speed limit (rate-of-change) for how fast Lambda can provision new execution environments during an instantaneous spike.
- **Behavior:**
  - Suppose Function A is sitting idle at 0 RPS, and suddenly receives **1,000 requests in a single second** with a duration of 1 second (1,000 instantaneous concurrency demanded).
  - Even though your Regional limit is 1,000, Function A is limited by two checks:
    1. **Its Reserved Concurrency limit (200):** It will immediately cap at 200 regardless of burst capacity.
    2. **Burst check (if it had no reservation):** If an unreserved function spikes from 0 to 5,000 concurrency instantly, Lambda only scales immediately up to its initial burst quota (e.g., 1,000 or 3,000 depending on the region). Any excess requests arriving faster than the scaling rate are throttled until additional environments are provisioned in subsequent 10-second intervals.



Section 4: Invoking Lambda functions.

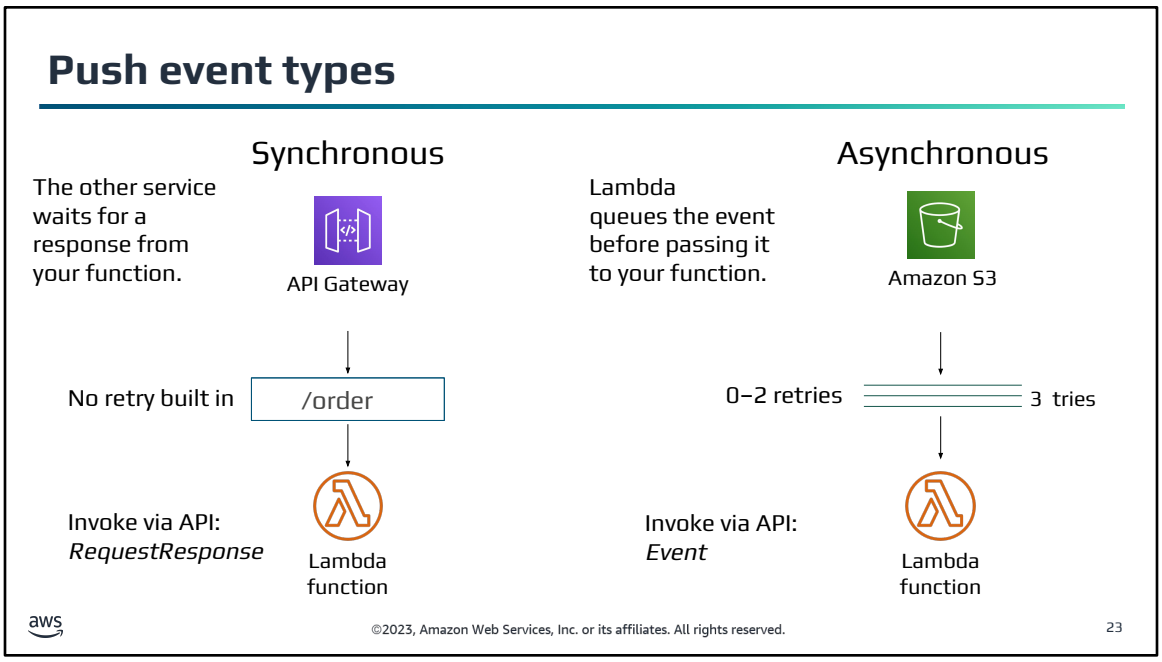
Push and pull models



aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Each event source invokes a Lambda function by using either a push or a pull model:

- *Push model* – An event source directly invokes the Lambda function when the event occurs.
- *Pull (or polling) model* – An event source puts information into a stream or queue. Lambda polls the stream or queue and invokes your Lambda function when it detects an event.



Some services invoke your function directly.

For *synchronous* invocation, the other service waits for the response from your function. Consider an example where API Gateway is the event source. In this case, when a client makes a request to your API, that client expects a response immediately. This model has no built-in retry in Lambda. You must manage your retry strategy in your application code.

Examples of synchronous event sources include the following:

- Elastic Load Balancing
- Amazon Cognito
- Amazon Lex
- Amazon Alexa
- Amazon API Gateway
- AWS CloudFormation
- Amazon CloudFront
- Amazon Kinesis Data Firehose

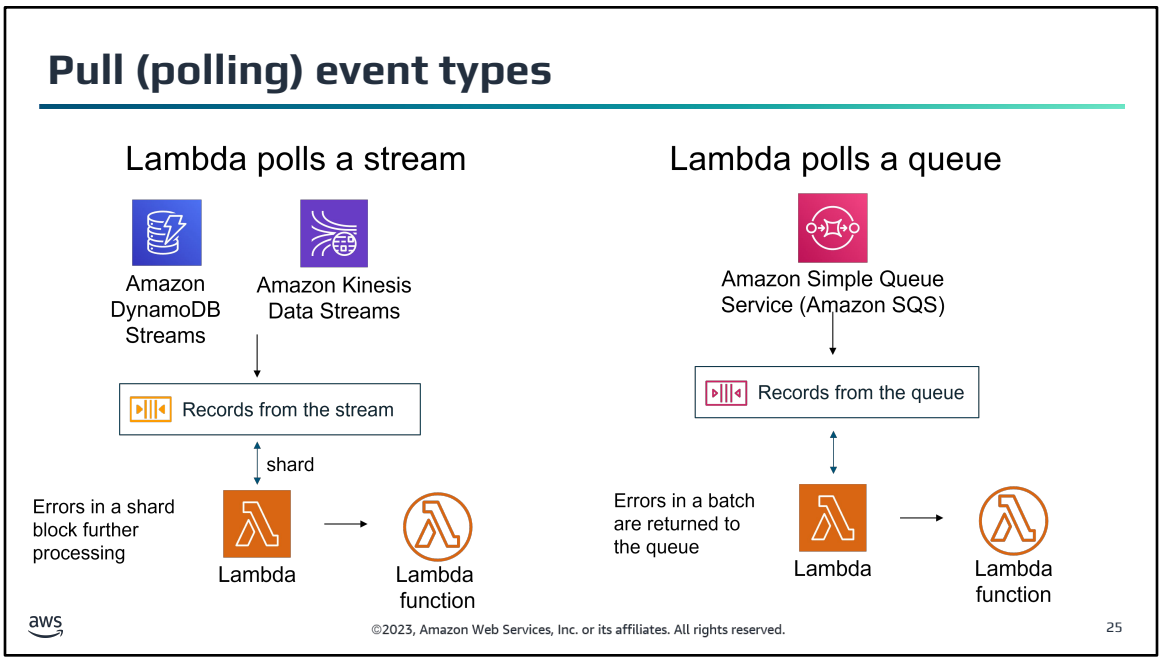
For *asynchronous* invocation, Lambda queues the event before passing it to your function. The other service gets a success response as soon as the event is queued and isn't aware of what happens afterwards. The event sources "sends it then forgets it". If an error occurs, Lambda will automatically retry the invocation twice. It can send failed events to a dead-letter queue that you configure.

Examples of asynchronous event sources include the following:

- Amazon S3
- Amazon SNS
- Amazon Simple Email Service (Amazon SES)
- AWS CloudFormation
- Amazon CloudWatch Logs
- Amazon CloudWatch Events
- AWS Config

In both cases, you invoke your Lambda function by using the *invoke* operation, and you can specify the invocation type as synchronous or asynchronous. When you use an AWS services as a trigger, the invocation type is predetermined for each service. You have no control over the invocation type that these event sources use when they invoke your Lambda function.

For more information how to invoke Lambda functions, see “Invoking Lambda functions” at https://docs.aws.amazon.com/lambda/latest/dg/invoking-lambda-functions.html.



In the polling model, Lambda uses event source mappings to get information from a stream or queue. Lambda polls the stream or queue. If it finds records, it delivers the payload and invokes the Lambda function. In this model, the Lambda service itself is extracting data from the stream or queue for processing by the Lambda function.

- Stream-based event sources include Amazon DynamoDB Streams and Amazon Kinesis Data Streams. Stream records are organized into shards. Lambda polls the stream for records and attempts to invoke the function. If a failure occurs, Lambda will not read any new shards until the failed batch of records expires or is processed successfully. You can configure error handling options to modify how Lambda deals with errors.
- Queue-based event sources include Amazon Simple Queue Service (Amazon SQS). Lambda polls the queue for records. If the invocation fails or times out, then the failed message is returned to the queue. Lambda continues to retry the failed message until it's processed successfully or the retry limit or message retention period is reached. Failed messages can be sent to an OnFailure destination or a dead-letter queue.

You can create a mapping between an event source and your Lambda function. Lambda reads items from the event source and invokes the function. For more information about the *CreateEventSourceMapping* operation, see "CreateEventSourceMapping" at https://docs.aws.amazon.com/lambda/latest/dg/API_CreateEventSourceMapping.html.

Polling does not incur any charge. You are charged only if actions result from a poll.

With streams, Lambda retries until it succeeds or the record reaches a 24-hour expiration. Thus, an erroring record can block processing. You can configure error handling options to bypass failing records and send them to an OnFailure destination for offline processing. For more information about how Lambda works with streams, see “Using AWS Lambda with Amazon Kinesis” at https://docs.aws.amazon.com/lambda/latest/dg/with-kinesis.html.

With queues, failed records become visible on the queue to be resent to your Lambda function up to a number of retries, which you can configure. You can send records that continue to fail to a dead-letter queue. For more information about how Lambda works with queues, see “Using AWS Lambda with Amazon SQS” at https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html.

Invoking a function synchronously

Invoke

```
aws lambda invoke \
--function-name my-function \
--payload '{ "name": "Bob" }' \
response.json
```

Response

```
{"ExecutedVersion": "$LATEST","StatusCode": 200}
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

This example shows the command line code to invoke a function synchronously and illustrates the format of the response that Lambda would return.

Notice that with a synchronous event, the acknowledgement is the 200 status.

Invoking a function asynchronously

Invoke

```
aws lambda invoke \
--function-name my-function \
--invocation-type Event \
--payload '{ "name": "Bob" }' \
response.json
```

Ack

```
{"StatusCode": 202}
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

This example shows the command line code to invoke a function asynchronously and illustrates the format of the response that Lambda would return. Notice that with an asynchronous event, the acknowledgement is the 202 status, which indicates that the request was received.

Section 4 key takeaways

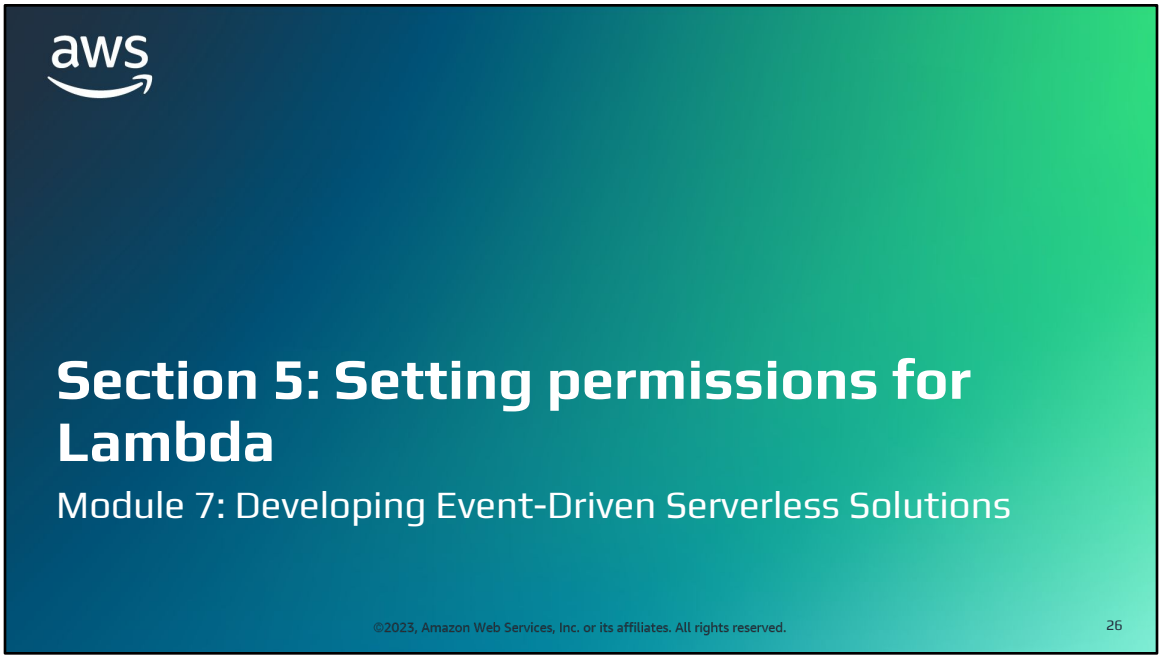


- Lambda invokes Lambda functions by using one of these invocation models:
  - Synchronous
  - Asynchronous
  - Polling (event source mapping)
- Lambda has built-in handling that differs by event source type.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

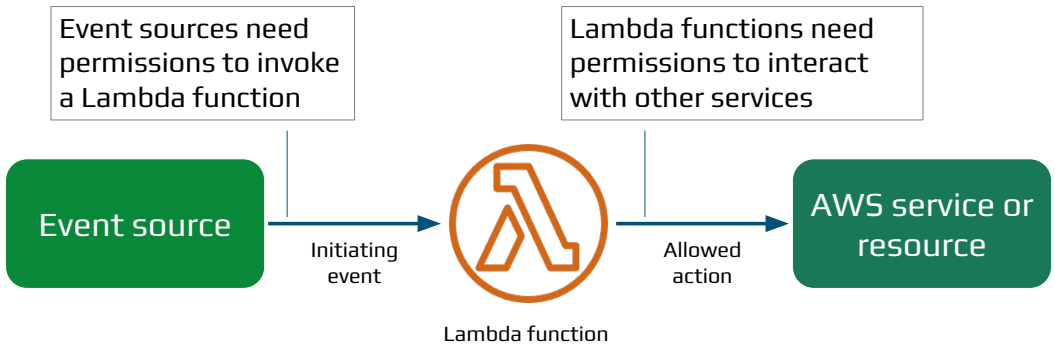
Some key takeaways from this section of the module include:

- AWS Lambda uses either a push or pull model for invocation
- Push – Can be synchronous or asynchronous
- Pull – Can be stream-based or non-stream-based



Section 5: Setting permissions for Lambda.

Lambda permissions



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

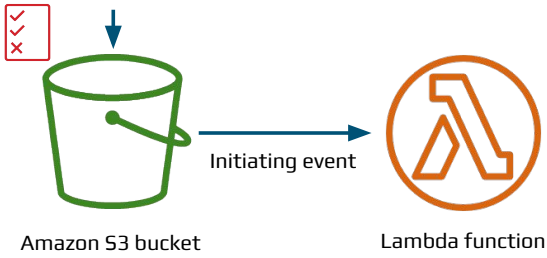
Lambda requires two types of permissions. First, event sources need permissions to invoke a Lambda function. Next, Lambda functions need permissions to interact with other AWS services and resources.

Permissions are handled through IAM.

For more information, see “Lambda resource access permissions” at https://docs.aws.amazon.com/lambda/latest/dg/lambda-permissions.html.

Invocation permissions

An IAM resource policy gives permissions to invoke the function.



Resource policy:

- Associated with synchronous or asynchronous event source
- Allows the event source to take the *lambda:InvokeFunction* action

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

An *IAM resource policy* tells the Lambda service which push event sources have permissions to invoke the Lambda function. Resource policies also make it easy to grant access to the Lambda function across AWS accounts. For example, suppose that you need an Amazon S3 bucket in account A to invoke your Lambda function in account B. You can create a resource policy that allows account A to invoke the function in account B.

Notice that the IAM resource policy is on the bucket, not on the Lambda function.

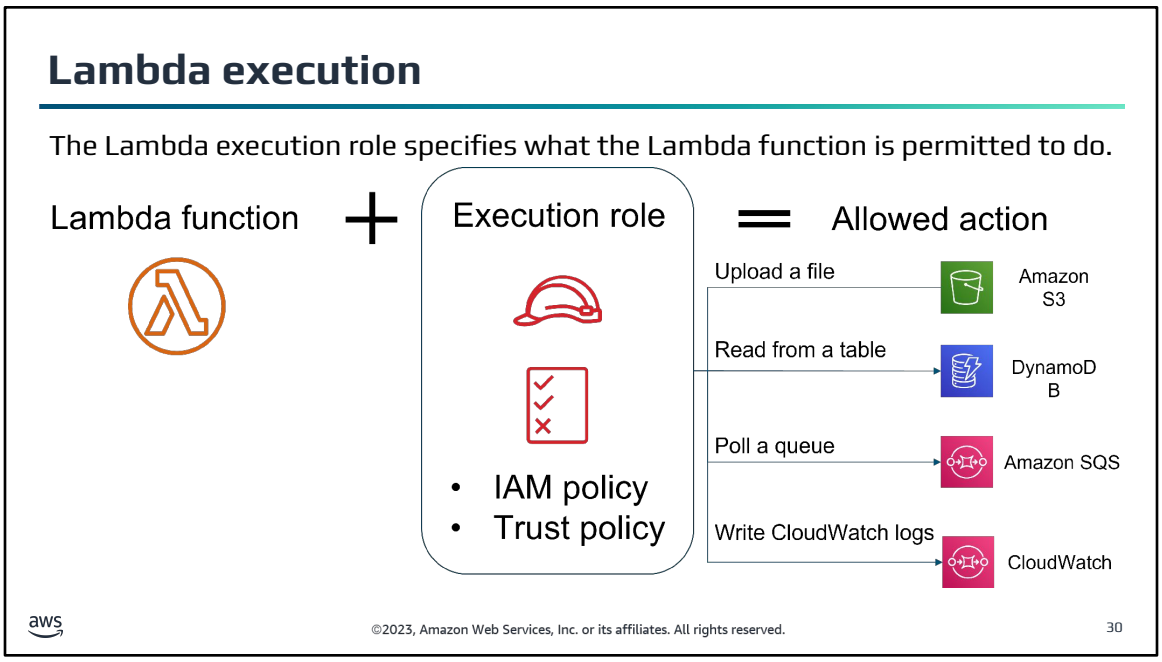
For more information, see “Using Resource-Based Policies for AWS Lambda” at https://docs.aws.amazon.com/lambda/latest/dg/access-control-resource-based.html.

Example resource (function) policy

```
{
    "Version": "2012-10-17",
    "Id": "default",
    "Statement": [
        {
            "Sid": "lambda-fd269e28-988b-4d2b-96ae-eabcd7dc399c",
            "Effect": "Allow",
            "Principal": {
                "Service": "s3.amazonaws.com"
            },
            "Action": "lambda:InvokeFunction",
            "Resource": "arn:aws:lambda:function:myFirstFunction",
            "Condition": {
                "ArnLike": {
                    "AWS:SourceARN": "arn:aws:s3:::myBucket1"
                }
            }
        }
    ]
}
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

In this example, this resource policy on an S3 bucket gives Amazon S3 permissions to invoke the Lambda function that is called *myFirstFunction*.



The *Lambda execution role* grants your Lambda function permissions to access AWS services and resources. You select or create the execution role when you create a Lambda function.

Two policies are attached to the execution role:

- *IAM policy* – Defines the actions that the Lambda function can take on another AWS service or resource, such as writing to a DynamoDB table
- *Trust policy* – Allows the Lambda service to assume the execution role

To grant permissions to Lambda to use the AssumeRole action, you must have permissions for the *iam:PassRole* action.

Polling event sources also need the execution role to give the Lambda function permissions to read from the queue or stream.

The Lambda function also needs the execution role to poll event sources to be able to read from the queue or stream.

For more information, see “Lambda execution role” at https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html.

Execution role example

IAM policy

```
{
    "Effect": "Allow",
    "Action": [
           "logs:CreateLogGroup",
           "logs:CreateLogStream",
           "logs:PutLogEvents"
       ],
    "Resource": "arn:aws:logs:*:*:*"
}
```

Trust policy

```
{
    "Effect": "Allow",
    "Principal": {
       "Service": "lambda.amazonaws.com"
      },
    "Action": "sts:AssumeRole"
}
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

IAM policies are associated with a role that another entity, such as Lambda or another resource, can assume. Policies aren't directly attached to a Lambda function.

Some examples of the relevant policy lines for a Lambda execution role include:

- In the example, the IAM policy allows your Lambda function to perform some CloudWatch Logs actions. These actions include creating a log group and log stream, and writing to the log stream.
- In the example, the trust policy gives the Lambda service permissions to assume the role and invoke the Lambda function on your behalf.

Adding permissions by using the AWS CLI

Give Amazon SNS permissions to invoke a function

```
aws lambda add-permission \
--function-name my-function \
--action lambda:InvokeFunction \
--statement-id sns \
--principal sns.amazonaws.com
```

Output

```
{
"Statement":
{
"Sid":"sns",
"Effect":"Allow",
"Principal":{
"Service":"sns.amazonaws.com"
},
"Action":"lambda:InvokeFunction",
"Resource":"arn:aws:lambda:us-east-2:123456789012:function:my-function"
}
}
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

This example shows the AWS Command Line Interface (AWS CLI) code to give Amazon SNS permissions to invoke a function and shows the IAM policy statement that results.

Section 5 key takeaways



- Lambda functions require permissions to be invoked and permissions to interact with other services.
- Both types of permissions are defined by using IAM or trust policies.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Some key takeaways from this section of the module include:

- Lambda functions require permissions to be invoked and permissions to interact with other services.
- Both types of permissions are defined by using IAM or trust policies.